

# Programmazione 1

## Corso c

09/10/2023

>while e for: il loop in C

Alessandro Mazzei

[alessandro.mazzei@unito.it](mailto:alessandro.mazzei@unito.it)

Dipartimento di Informatica

Università degli Studi di Torino



**UNIVERSITÀ**  
**DI TORINO**

# Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

# Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

# Il costrutto while

- Il *costrutto di iterazione* (ciclo)

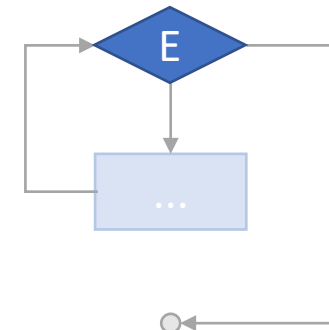
```
while (<espressione E>) {
```

...

```
}
```

...

valuta l'espressione E:



# Il costrutto while

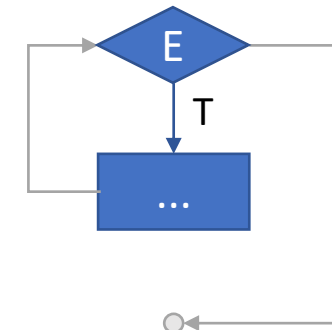
- Il *costrutto di iterazione* (ciclo)

```
while (<espressione E>) {  
    ...  
}  
...
```



valuta l'espressione E:

- se E è vera, il blocco viene eseguito ...



# Il costrutto while

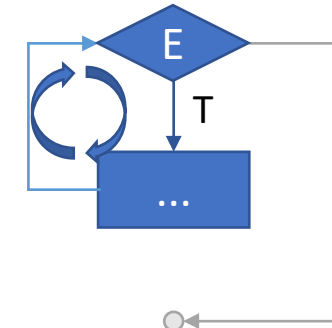
- Il *costrutto di iterazione* (ciclo)

```
while (<espressione E>) {  
    ...  
}  
...
```



valuta l'espressione E:

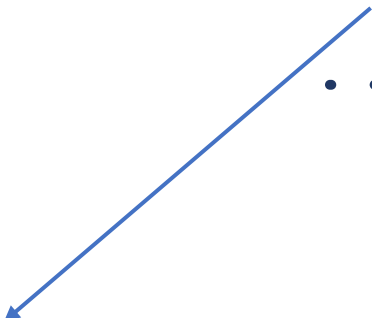
- se E è vera, il blocco viene eseguito ed il controllo torna al while()



# Il costrutto while

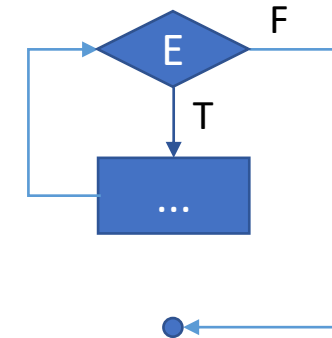
- Il *costrutto di iterazione* (ciclo)

```
while (<espressione E>) {  
    ...  
}  
...
```

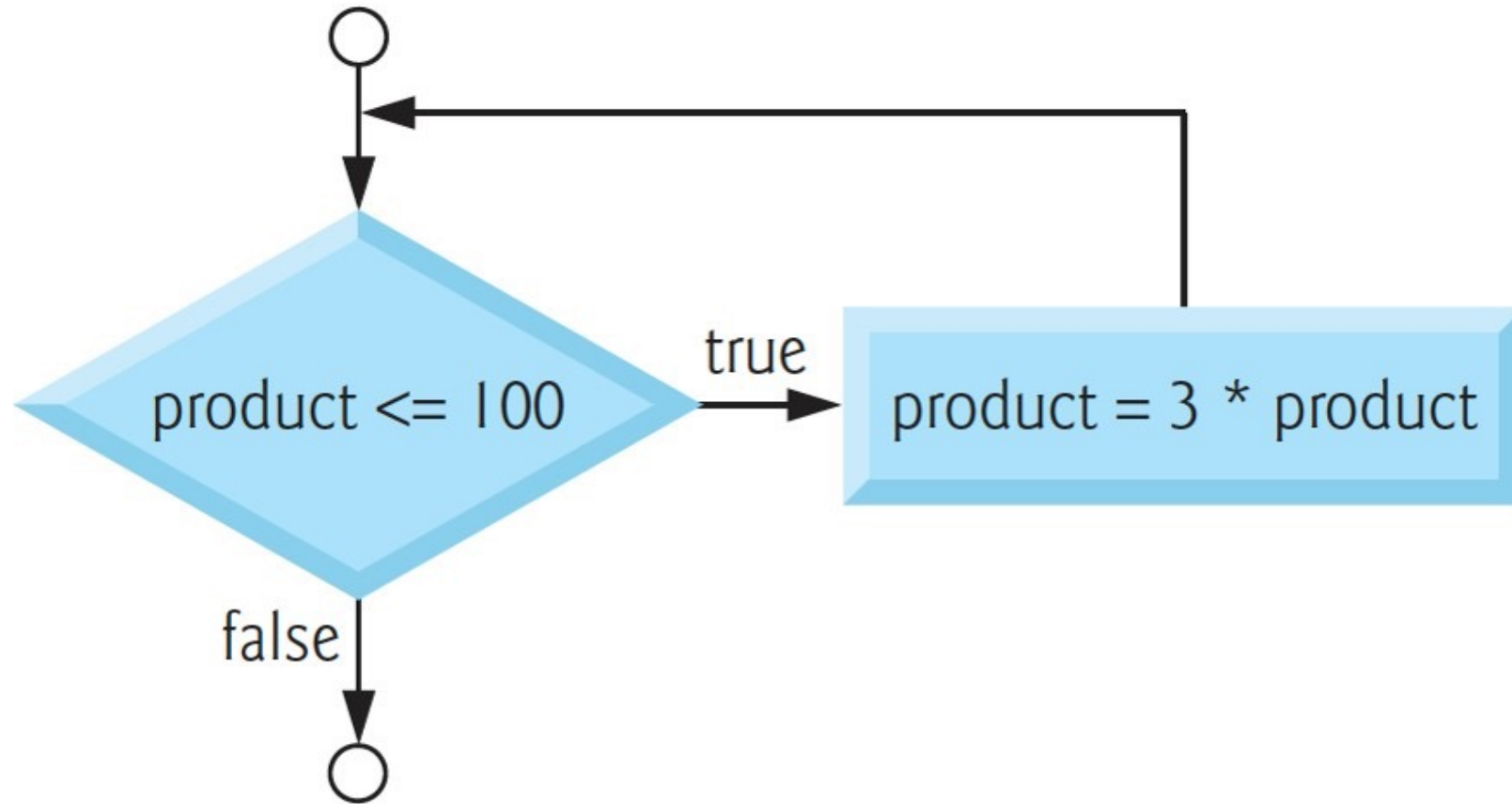


valuta l'espressione E:

- se E è vera, il blocco viene eseguito ed il controllo torna al while()
- Altrimenti, il controllo passa oltre la fine del blocco



# While flowchart





# Il costrutto while

- L'espressione E può essere una qualsiasi espressione:

```
char input;  
printf("Premi un tasto:");  
scanf(" %c",&input);  
while (input != 'z') {  
    printf("Sbagliato!\nPremi un tasto:");  
    scanf(" %c",&input);  
}
```

# Il costrutto while

- L'espressione E, spesso, è il confronto fra una variabile indice *i* ed un valore costante

```
int i = 0;  
while (i < n) {  
    ...  
}
```

# Il costrutto while

- L'indice *i* in tal caso è tipicamente aggiornato (es: incrementato di una unità) ad ogni iterazione nel corpo del ciclo

```
int i = 0;  
while (i < n) {  
    ...  
    i = i+1;  
}
```

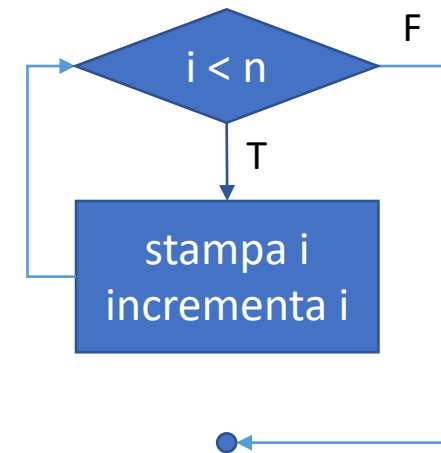
# Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

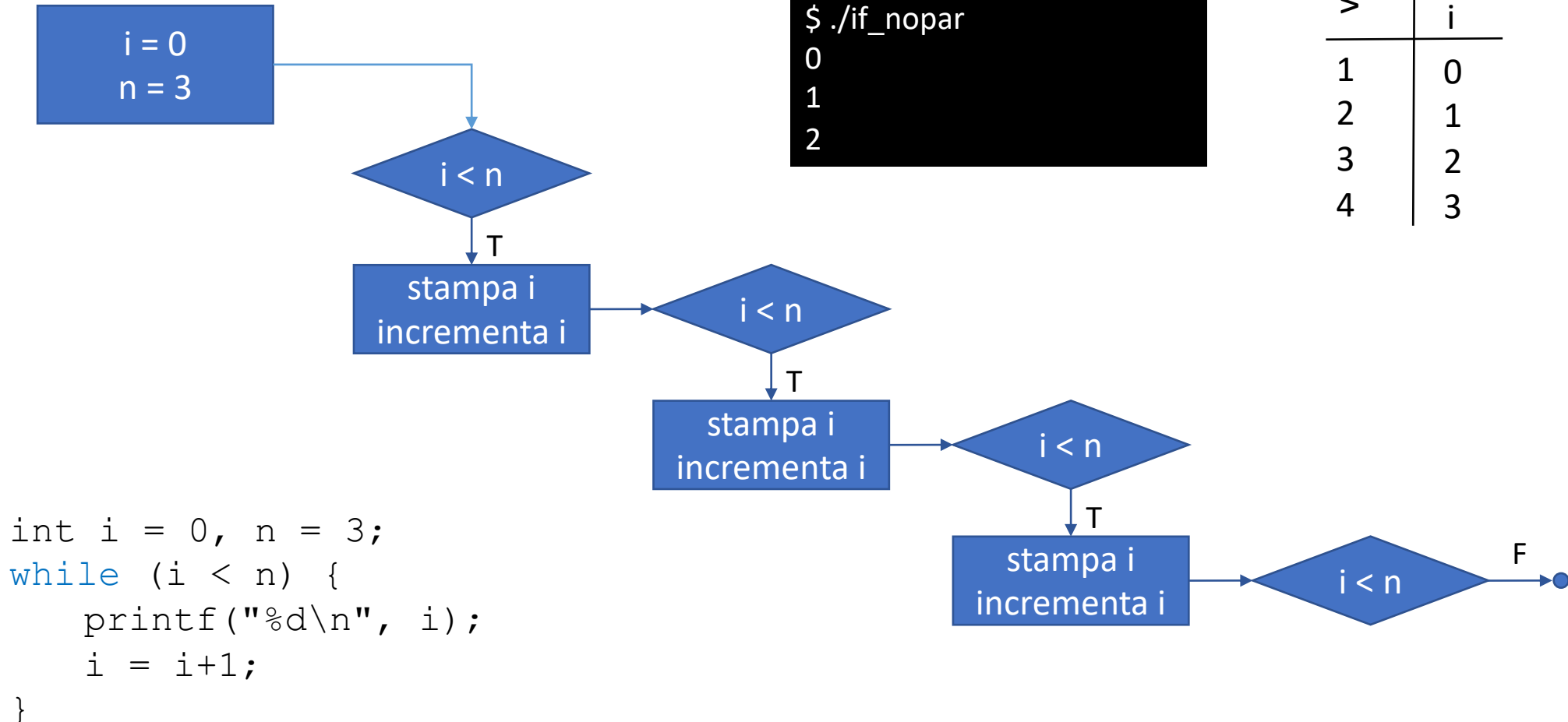
# Il costrutto while - Esempio

- Stampare i primi tre numeri naturali

```
int i = 0, n = 3;  
while (i < n) {  
    printf("%d\n", i);  
    i = i+1;  
}
```



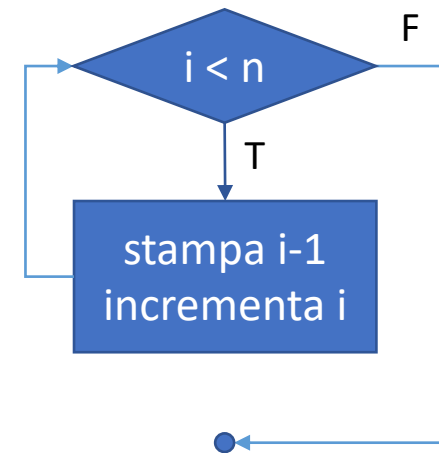
# Il costrutto while - Esempio



# Il costrutto while - Esempio

- Stampare i primi tre numeri naturali

```
int i = 1, n = 3;  
while (i <= n) {  
    printf("%d\n", i);  
    i = i+1;  
}
```

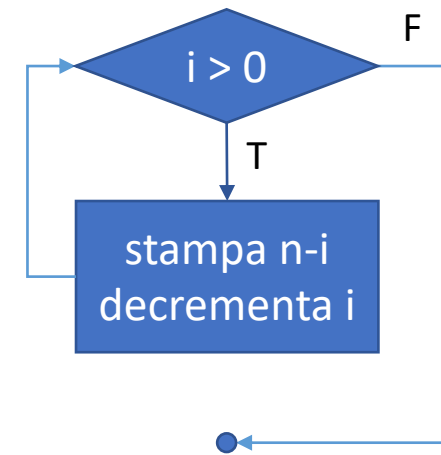


- Avremmo anche potuto inizializzare  $i$  a 1 e valutare nel *while* l'espressione  $i \leq n$

# Il costrutto while - Esempio

- Stampare i primi tre numeri naturali

```
int i = 3, n = 3;  
while (i > 0) {  
    printf("%d\n", n-i);  
    i = i-1;  
}
```



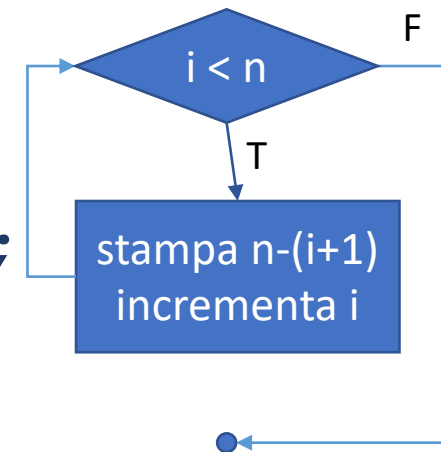
- Avremmo anche potuto decrementare  $i$  ad ogni ciclo, definendo  $i$  ed  $n$  a 3 ed aggiornando printf()



# Il costrutto while - Esempio

- Stampare i primi tre numeri naturali in ordine decrescente

```
int i = 0, n = 3;  
while (i < n) {  
    printf("%d\n", n - (i + 1));  
    i = i + 1;  
}
```

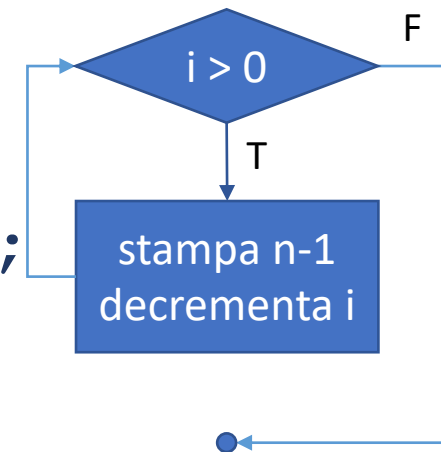


```
2  
1  
0
```

# Il costrutto while - Esempio

- Stampare i primi tre numeri naturali in ordine decrescente

```
int i = 3, n = 3;  
while (i > 0) {  
    printf("%d\n", n-1);  
    i = i-1;  
}
```



```
2  
1  
0
```

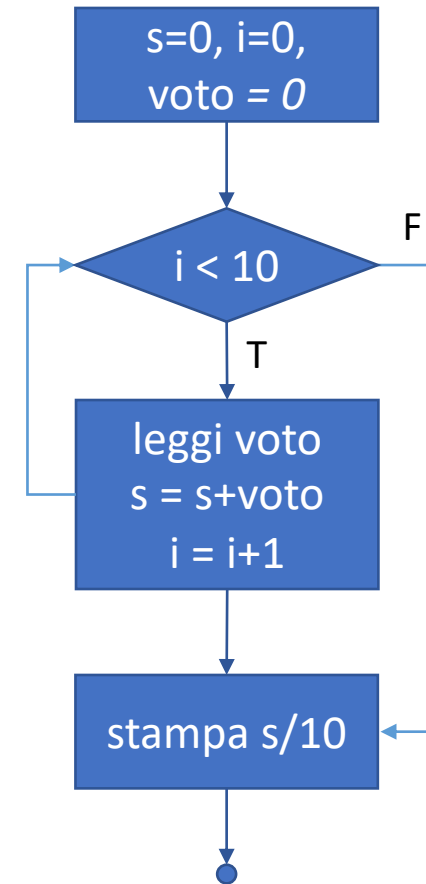
# Il costrutto while

- Nell'esempio precedente l'indice di iterazione  $i$  conteneva l'informazione di interesse per l'utente
- Più comunemente, l'indice serve unicamente a contare i cicli eseguiti, mentre l'informazione di interesse per l'utente è codificata in altre variabili

# Esercizio

- Calcolare la media di 10 voti inseriti dall'utente

```
int main(void) {  
    int s = 0, i = 0, voto = 0;  
    while (i < 10) {  
        printf("Inserisci voto: ");  
        scanf("%d", &voto);  
        s = s + voto;  
        i = i + 1;  
    }  
    printf("Voto medio: %f\n", s/(float)10);  
}
```



# Esercizio

- Stampare  $m^n$  senza utilizzare la funzione *pow()* di *math.h*

```
int m=4, n=3, p = 1, i = 0;
while (i < n) {
    p = p * m;
    i = i + 1;
}
printf ("%d^ %d uguale %d\n", m, n, p);
```

# Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

# Variabili sentinella

- Negli esempi precedenti il ciclo era eseguito esattamente un numero di volte noto già alla prima valutazione di E -> *variabile contatore*
- È comune dover eseguire un ciclo per la prima volta senza sapere se dovrà essere ripetuto e quante volte
- È usuale l'uso di una *variabile sentinella*, ovvero variabile booleana valutata ad ogni ingresso nel ciclo

# Variabili sentinella

- Calcolare la media di un numero arbitrario di voti inseriti dall'utente
- L'utente inserisce un voto negativo per indicare la fine dell'input
- Eseguire il ciclo finchè la variabile sentinella *eseguiCiclo* è true
- Come inizializzare *eseguiCiclo* perchè il primo ciclo sia eseguito ?
- Quando e come aggiornare *eseguiCiclo* ?



# Variabili sentinella

- Variabili input: totale  $s$ , contatore  $i$ , sentinella *eseguiCiclo*
- *eseguiCiclo* impostata a true prima del primo ciclo
- Ad ogni ciclo, leggere un voto
  - se il voto è positivo, sommarlo ad  $s$ , incrementare  $i$  e lasciare che sia eseguito un altro ciclo
  - altrimenti, impostare *eseguiCiclo* a false per impedire l'esecuzione di un altro ciclo
- Infine, calcolare il voto medio come  $s / i$  (senza troncature)

# Variabili sentinella

- Variabili input: totale  $s$ , contatore  $i$ , sentinella *eseguiCiclo*
- *eseguiCiclo* impostata a true prima del primo ciclo
- Ad ogni ciclo, leggere un voto
  - se il voto è positivo, sommarlo ad  $s$ , incrementare  $i$  e lasciare che sia eseguito un altro ciclo
  - altrimenti, impostare *eseguiCiclo* a false per impedire l'esecuzione di un altro ciclo
- Infine, calcolare il voto medio come  $s / i$  (senza troncature)

*if-else*  
nel ciclo

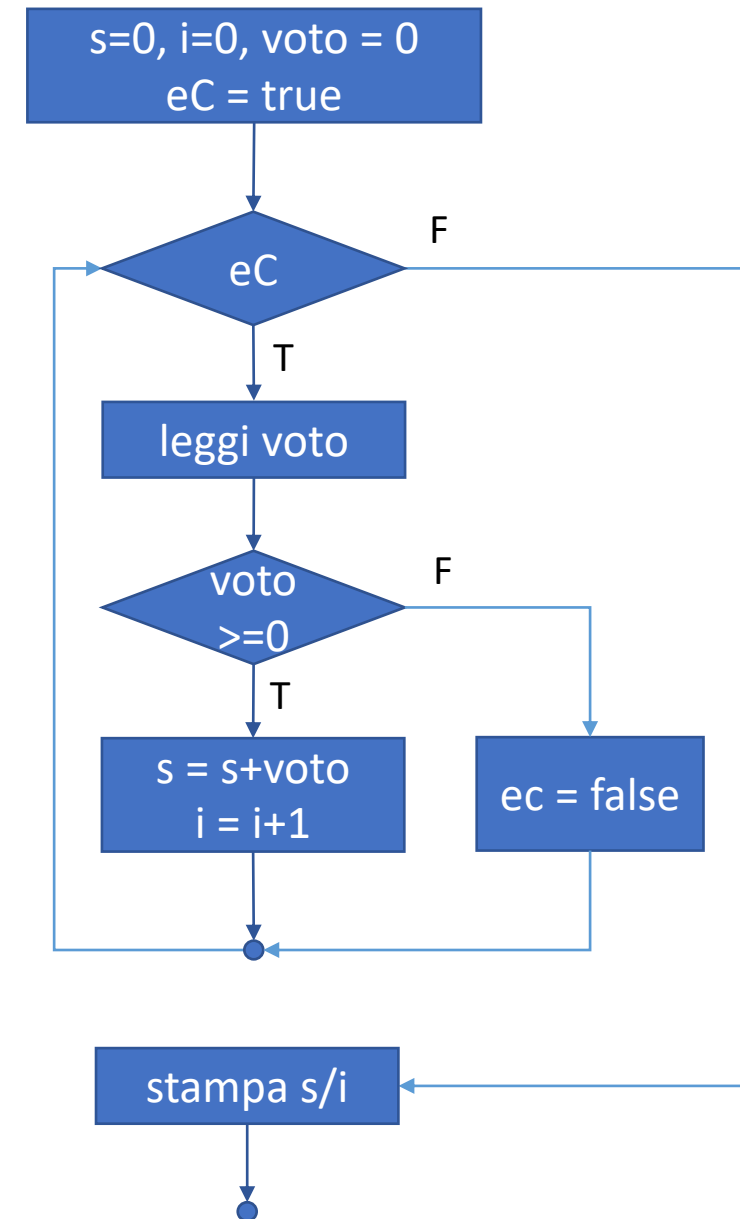


# Variabili sentinella

```
int main(void) {  
    int s = 0, i = 0, voto = 0;  
    int eseguiCiclo = true;  
    while (eseguiCiclo) {  
        printf("%s", "Inserisci voto: ");  
        scanf("%d", &voto);  
        if (voto >= 0) {  
            s = s + voto;  
            i = i + 1;  
        }  
        else {  
            eseguiCiclo = false;  
        }  
    }  
    printf("Voto medio: %f\n", s/(float)i);  
}
```

esecuzione  
ciclo  
legata a  
*eseguiCiclo*

indice *i*  
usato per  
media



# Variabili sentinella

- Nel libro di testo il problema viene risolto senza variabile sentinella:
  - acquisendo il primo voto al di fuori del ciclo while
  - condizionando l'ingresso nel ciclo a voto > 0
- Soluzione corretta ma duplica il codice per l'input utente
- Noi preferiremo soluzioni basate su allocazione di una variabile sentinella perchè più flessibile e rende il codice è più leggibile

# Variabili sentinella

- Come nell'esercizio precedente, calcolare la media di un numero arbitrario di voti inseriti dall'utente **limitato a 10 voti**
- La condizione di ingresso nel ciclo si complica, poiché dipende dalla sentinella e dal contatore:

Contatore: `i < 10`

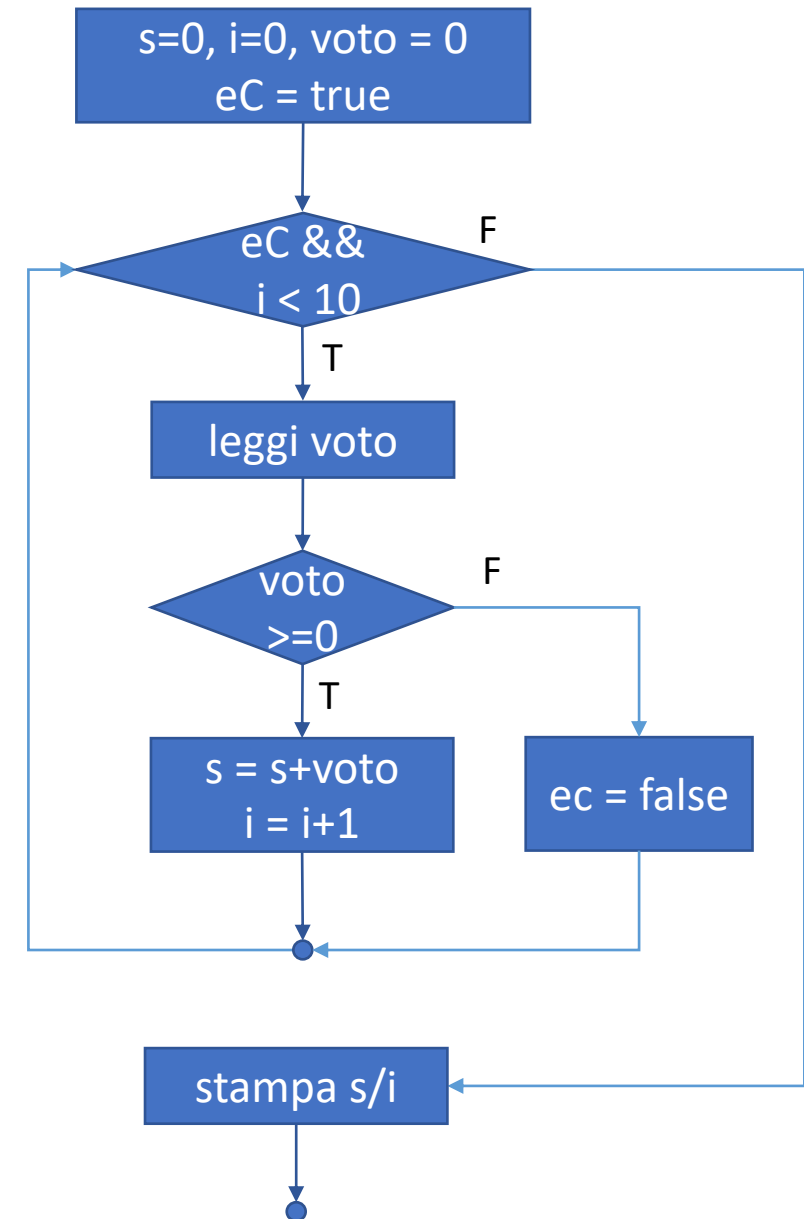
`&&`

Sentinella: `eseguiCiclo == true`

# Variabili sentinella

```
int main(void) {  
    int s = 0, i = 0, voto = 0;   
    int eseguiCiclo = true;  
    while (i < 10 && eseguiCiclo) {  
        printf("%s", "Inserisci voto: ");  
        scanf("%d", &voto);  
        if (voto >= 0) {  
            s = s + voto;  
            i = i + 1;  
        }  
        else {  
            eseguiCiclo = false;  
        }  
    }  
    printf("Voto medio: %f\n", s/(float)i);  
}
```

espressione complessa



# Variabili sentinella

- Come nell'esercizio precedente, calcolare la media di un numero arbitrario di voti inseriti dall'utente limitato a 10 voti
- La condizione di ingresso nel ciclo si complica  
 $\text{contatore } i < 10 \quad \text{AND} \quad \text{sentinella eseguiCiclo} == \text{true}$

**ATTENZIONE:** l'ingresso e l'uscita dai cicli (anche annidati) è una delle cause più frequenti di errore all'esame

# Il ciclo do-while

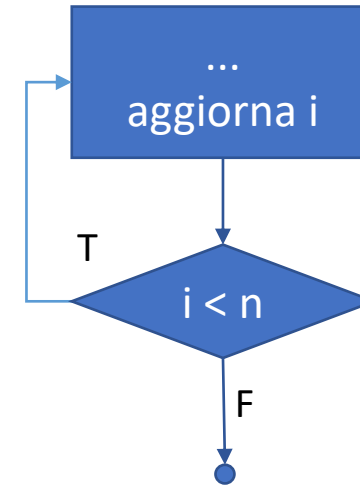
- Il ciclo *do-while* è una variante di *while* dove
  1. il corpo del *while* viene eseguito (almeno 1 volta!)
  2. viene valutata l'espressione  $\langle E \rangle$
  3. Se vera si torna su ...

do {

...

$i = i + 1;$

} while ( $i \leq n$ )



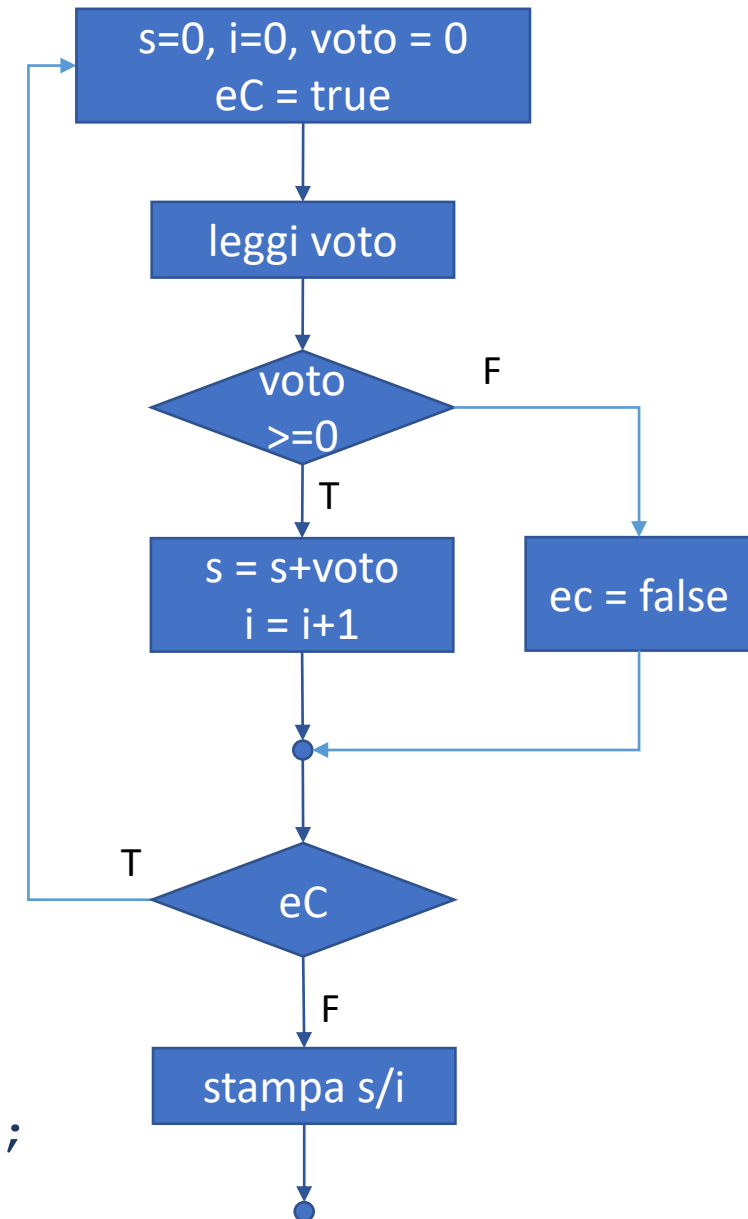


# Il ciclo do-while: esempio

```
int main(void) {  
    int s = 0, i = 0, voto = 0;  
    int eseguiCiclo = true;  
    do {  
        printf("Inserisci voto: ");  
        scanf("%d", &voto);  
        if (voto >= 0) {  
            s = s + voto; i = i + 1;  
        }  
        else {  
            eseguiCiclo = false;  
        }  
    } while (eseguiCiclo);  
    printf("Voto medio: %f\n", s/(float)i);  
}
```

esecuzione  
primo ciclo  
incondizionata

esecuzione  
altri cicli  
condizionata



# Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

# Il costrutto while - Esempio

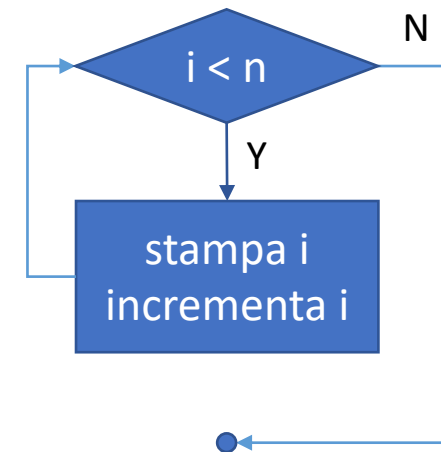
- Stampare i primi tre numeri naturali

```
int i = 0;  
n = 3;  
while (i < n) {  
    printf("%d\n", i);  
    i = i + 1;  
}
```

definizione variabile  
contatore o  
sentinella

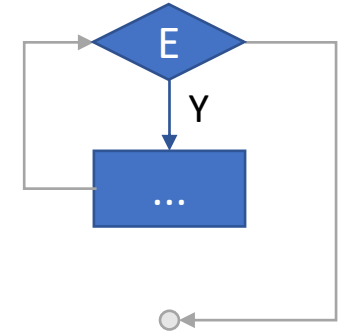
espressione  
ingresso ciclo

aggiornamento  
indice e/o sentinella



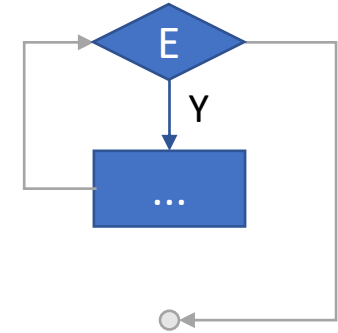
# Il costrutto for

- Il *costrutto di iterazione* (ciclo)



```
for (<inizializzazione>; <espressione E>; <aggiornamento>)  
{  
    . . .  
}
```

# Il costrutto for

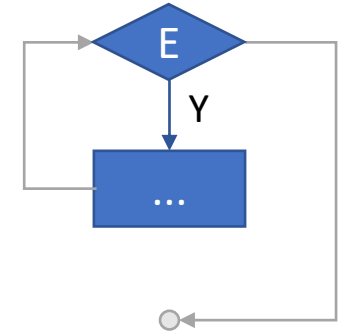


- Il *costrutto di iterazione* (ciclo)

```
for (<inizializzazione>; <espressione E>; <aggiornamento>)  
{  
    . . .  
}
```

- Viene inizialmente eseguita l'istruzione di inizializzazione
- Eseguita una volta sola
- Tipicamente assegnamento (e definizione) di variabile contatore o sentinella

# Il costrutto for



- Il *costrutto di iterazione* (ciclo)

```
for (<inizializzazione>; <espressione E>; <aggiornamento>)  
{  
    . . .  
}
```

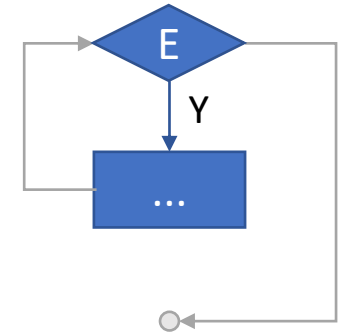
- L'espressione E viene valutata e, se vera, il ciclo viene eseguito
- Tipicamente confronto indice con intero e/o sentinella a true

# Il costrutto for

- Il *costrutto di iterazione* (ciclo)

```
for (<inizializzazione>; <espressione E>; <aggiornamento>)  
{  
    . . .  
}
```

- L'istruzione di aggiornamento viene eseguita al termine del blocco
- Tipicamente incremento contatore e/o aggiornamento sentinella

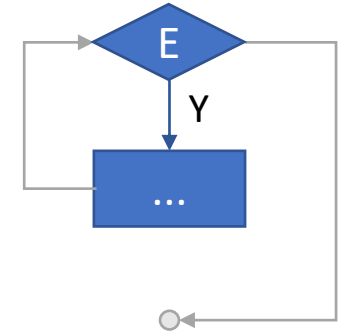


# FOR: esempio

```
int i, n = 3;  
for (i = 0; i < n; i = i+1) {  
    printf("%d\n", i);  
}
```



# Il costrutto for - Osservazioni

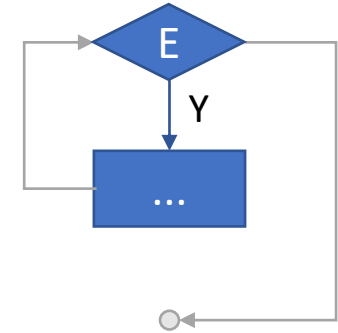


- L'esempio precedente è meglio scritto come

```
for (int i = 0 ; i < n ; i++) {  
    ...  
}
```

- L'inizializzazione consiste nella *definizione (dichiarazione + inizializzazione)* del contatore *i*
- Ma il contatore avrà **visibilità (scope) ristretta al corpo del ciclo**

# Il costrutto for - Osservazioni

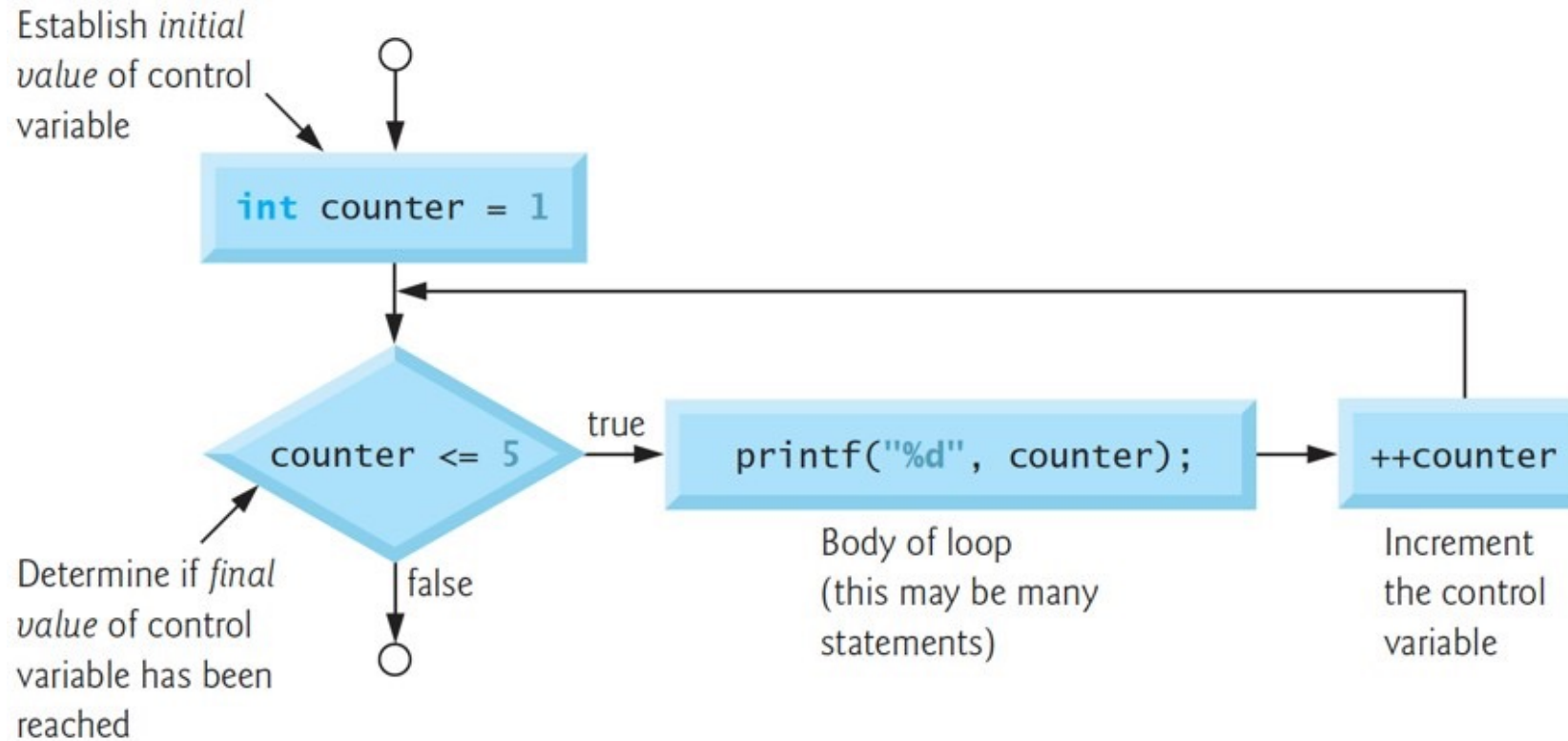


- L'esempio precedente può essere scritto come

```
for (int i = 0 ; i < n ; i++) {  
    ...  
}
```

- L'espressione  $i = i + 1$  è spesso sostituita da  $i++$  (*postincremento*)
- Utilizzeremo il postincremento solo in questo caso

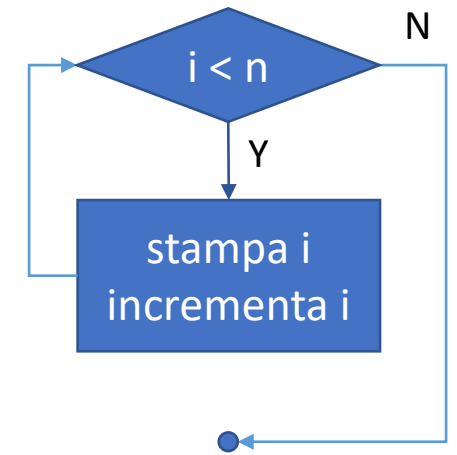
# FOR: flowchart



# Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

# Il costrutto for - Equivalenza



inizializzazione  
contatore o  
sentinella

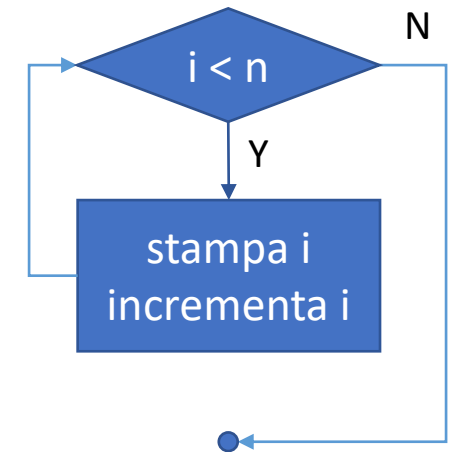
```
int i = 0, n = 3;
while (i < n) {
    printf("%d\n", i);
    i = i+1;
}
```

espressione  
ingresso ciclo

```
int i, n = 3;
for (i = 0; i < n; i = i+1)
{
    printf("%d\n", i);
}
```

aggiornamento  
indice e/o sentinella

# Il costrutto for - Equivalenza



```
int i = 0, n = 3;
while (i < n) {
    printf("%d\n", i);
    i = i+1;
}
```

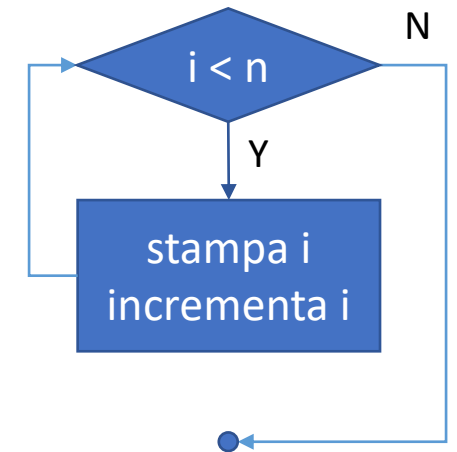
inizializzazione  
contatore o  
sentinella

```
int i=0, n = 3;
for (i=0; i < n; i = i+1)
{
    printf("%d\n", i);
}
```

espressione  
ingresso ciclo

aggiornamento  
indice e/o sentinella

# Il costrutto for - Equivalenza



inizializzazione  
contatore o  
sentinella

```
int i = 0, n = 3;
while (i < n) {
    printf("%d\n", i);
    i = i+1;
}
```

espressione  
ingresso ciclo

```
int i=0, n = 3;
for (i=0; i < n; i=i+1)
{
    printf("%d\n", i);
    i = i+1;
}
```

aggiornamento  
indice e/o sentinella

# Variabili sentinella

```
int main(void) {
    int s = 0, i = 0, voto = 0;
    bool eseguiCiclo = true;
    while (eseguiCiclo) {
        printf("%s", "Inserisci voto: ");
        scanf("%d", &voto);
        if (voto >= 0) {
            s = s + voto;
            i = i + 1;
        }
        else {
            eseguiCiclo = false;
        }
    } // end while
    printf("Voto medio: %f\n", s/(float)i);
}
```

Perchè  
dichiarazione  
non qui ?

Perchè  
aggiornamento  
non qui ?

```
int main(void) {
    int s = 0, i = 0, voto = 0;
    bool eseguiCiclo;
    for (eseguiCiclo = true; eseguiCiclo; )
        printf("%s", "Inserisci voto: ");
        scanf("%d", &voto);
        if (voto >= 0) {
            s = s + voto;
            i = i + 1;
        }
        else {
            eseguiCiclo = false;
        }
    } // end for
    printf("Voto medio: %f\n", s/(float)i);
}
```



# Variabili sentinella

```
int main(void) {
    int s = 0, i = 0, voto = 0;
    int eseguiCiclo = true;
    while (i < 10 && eseguiCiclo) {
        printf("%s", "Inserisci voto:");
        scanf("%d", &voto);
        if (voto >= 0) {
            s = s + voto;
            i = i + 1;
        }
        else {
            eseguiCiclo = false;
        }
    }
    printf("Voto medio: %f\n", s/(float)i);
}
```

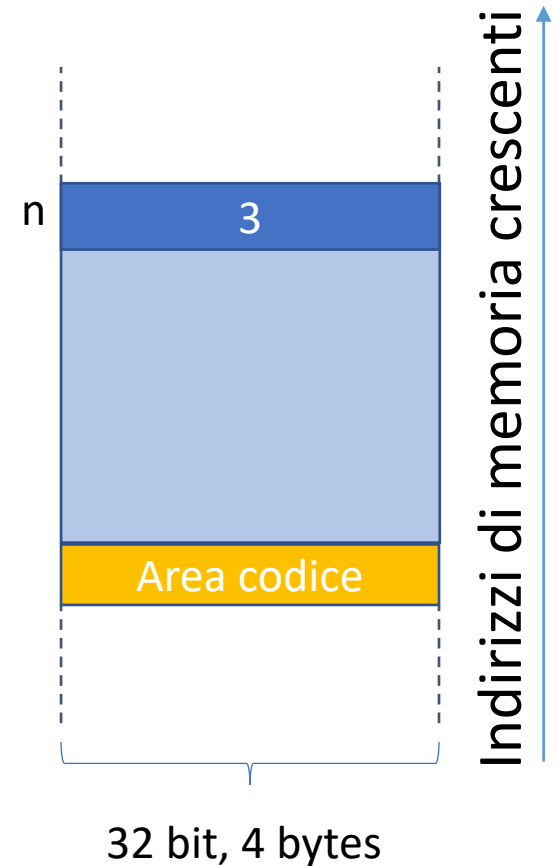
```
int main(void) {
    int s = 0, i = 0, voto = 0;
    for (i=0, bool eseguiCiclo=true; i < 10 &&
    eseguiCiclo; ) {
        printf("%s", "Inserisci voto: ");
        scanf("%d", &voto);
        if (voto >= 0) {
            s = s + voto;
            i = i + 1;
        }
        else {
            eseguiCiclo = false;
        }
    }
    printf("Voto medio: %f\n", s/(float)i);
}
```

# Scope delle variabili

- La variabile *i* ha scope limitato al blocco del for

```
void main() {  
    int n = 3;  
    for (int i = 0; i < n; i++) {  
        printf("%d\n", i);  
    }  
    printf("%d\n", i);  
}
```

\$/scope



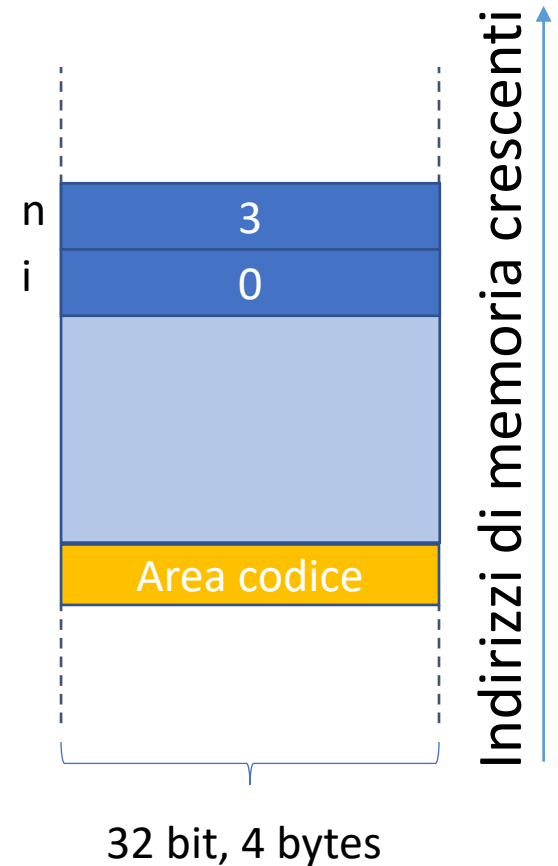
# Scope delle variabili

- La variabile *i* ha scope limitato al blocco del for

```
void main() {  
    int n = 3;  
    for (int i = 0; i < n; i++) {  
        printf("%d\n", i);  
    }  
    printf("%d\n", i);  
}
```



\$/scope



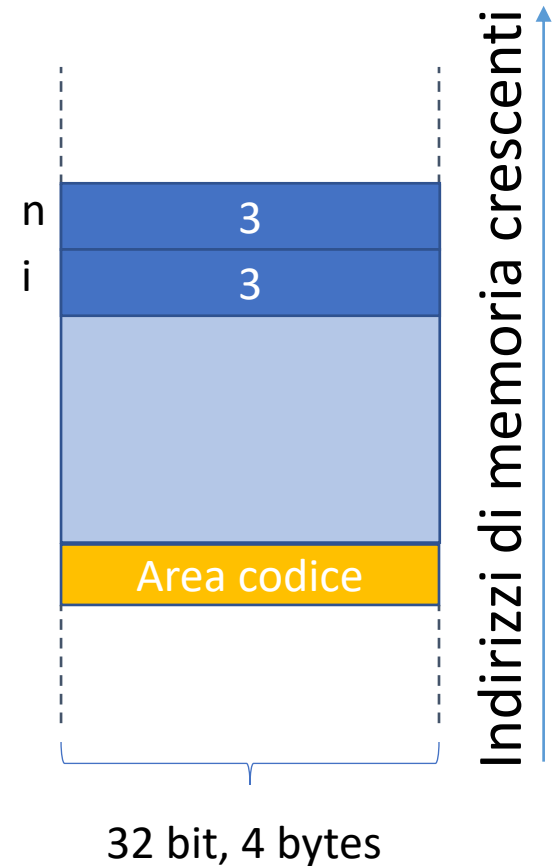
# Scope delle variabili

- La variabile *i* ha scope limitato al blocco del for

```
void main() {  
    int n = 3;  
    for (int i = 0; i < n; i++) {  
        printf("%d\n", i);  
    }  
    printf("%d\n", i);  
}
```



```
$/scope  
0  
1  
2
```



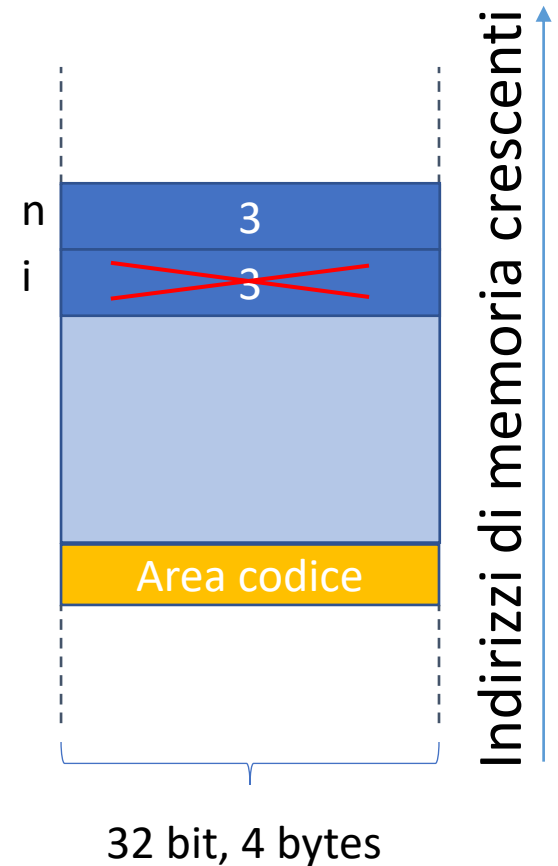
# Scope delle variabili

- La variabile *i* ha scope limitato al blocco del for

```
void main() {  
    int n = 3;  
    for (int i = 0; i < n; i++) {  
        printf("%d\n", i);  
    }  
    printf("%d\n", i);  
}
```



```
$/scope  
0  
1  
2
```

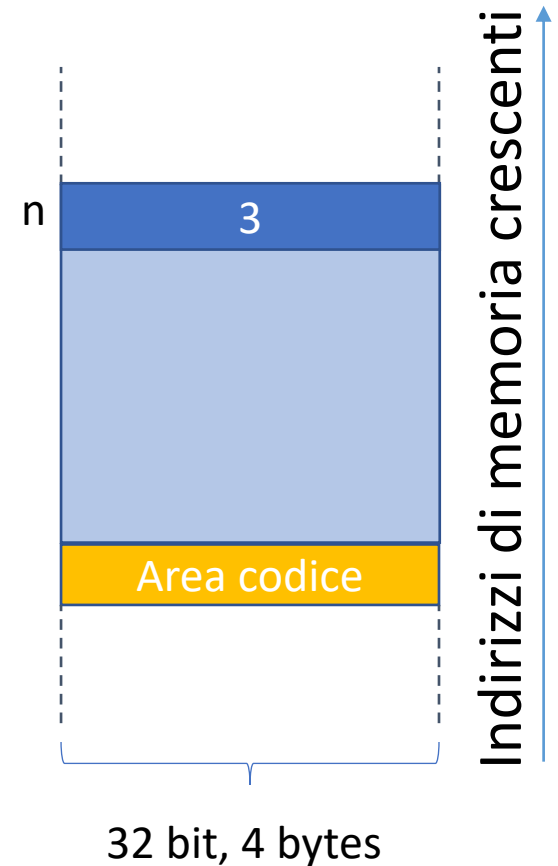


# Scope delle variabili

- La variabile *i* ha scope limitato al blocco del for

```
void main() {  
    int n = 3;  
    for (int i = 0; i < n; i++) {  
        printf("%d\n", i);  
    }  
    printf("%d\n", i);  
}
```

```
./scope  
0  
1  
2
```



# Scope delle variabili

- La variabile *i* ha scope limitato al blocco del for

```
void main() {  
    int n = 3;  
    for (int i = 0; i < n; i++) {  
        printf("%d\n", i);  
    }  
    printf("%d\n", i);  
}
```

```
$gcc scope.c -o scope  
scope.c: In function 'main':  
scope.c:7:17: error: 'i' undeclared (first use in this function)
```

